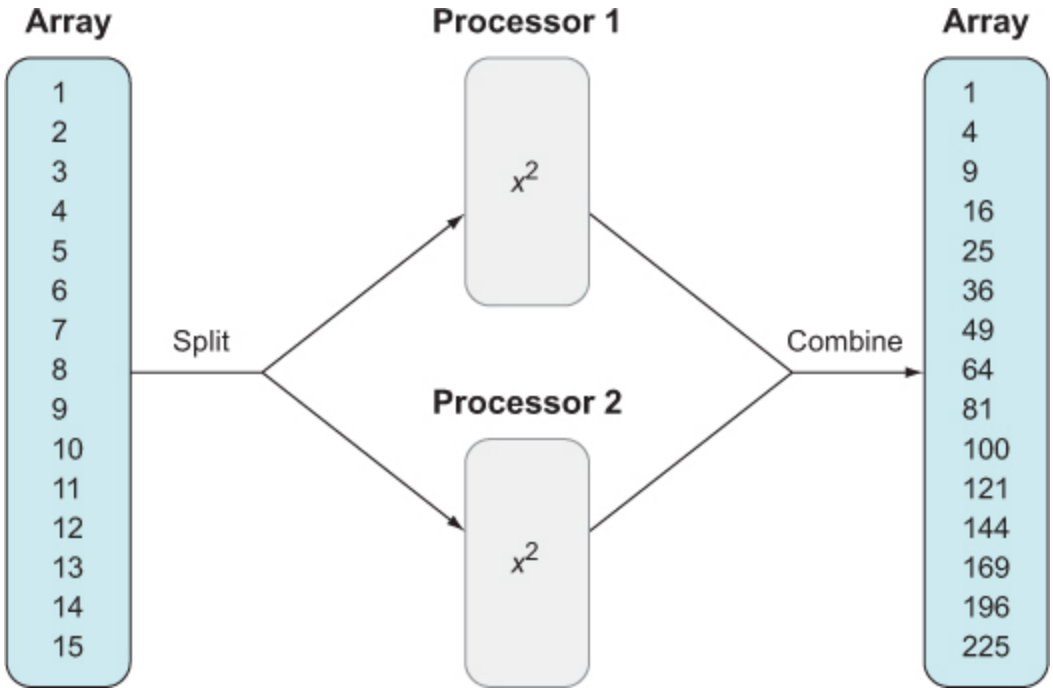
```
array([2304, 2401, 2500, 2601, 2704, 2809, 2916, 3025]),
array([3136, 3249, 3364, 3481, 3600, 3721, 3844, 3969])]
```

- ***1*** This function takes an array and squares each element.
- ***2*** Sets up an array with a sequence of numbers
- ***3*** Sets up a multiprocessing processor pool with 8 processes
- ***4*** Uses the pool's map function to apply the square function to each array in the list and returns the results in a list

Here we define a function, `square`, that takes an array and squares it. This is the function that will get distributed across multiple processes. We create some sample data that is simply the list of numbers from 0 to 63, and rather than sequentially squaring them in a single process, we chop up the array into 8 pieces and compute the squares for each piece independently on a different processor (figure 5.9).

Figure 5.9. A simple multiprocessing example. We want to more efficiently square all the numbers in an array. Rather than squaring each element one by one, we can split the array into two pieces and send each piece to a different processor that will square them simultaneously. Then we can recombine the pieces back into a single array.



You can see how many hardware processors your computer has by using the `mp.cpu_count()` function. You can see in listing 5.1 that we have 8.